Individual Assignment 2

Topic: Split, Ship and Operate Your Application: Services, Containers, CI/CD and Azure

Deadline: 2026-11-22 23:59

Written Comprehension Check: in class, in the first session after the deadline

Assignment Philosophy

The rules of Assignment 1 apply. AI use is allowed and expected; a passing pipeline and a responding URL are not, on their own, evidence of understanding. Grading weights the decisions you made and your ability to explain your system on paper, without notes.

In this assignment no infrastructure is provided. You write the Dockerfiles, the compose file, the pipeline and the deployment, and any line in those files may be asked about at the comprehension check.

Objective

Starting from the monolith built in Assignment 1:

1. **Split it** into separate services along the seam defined in Assignment 1 (§3 and ADR-2): one service per feature domain, each with its own container and its own data.
2. **Run it locally** with one command (`docker compose up`).
3. **Evolve its database** with versioned migrations that preserve existing data.
4. **Build, test and deploy it automatically**: every push is tested; every merge to `main` is deployed to **Azure** with no manual steps.
5. **Serve the UI as a static site** on Azure Static Web Apps, calling both services.
6. **Make it observable**: each service reports its own health.

The repository and use case stay the same. No new features are required.

1. Scope & Architecture Constraints

1a. Hard requirements:

- **Two services, one per feature domain.** Each service runs as its own process in its own container, built from its own Dockerfile. **Exactly two service containers.**
- **Static UI on Azure Static Web Apps.** The UI is static files (HTML/CSS/JS, or a framework built to static files) in `ui/`, deployed to **one Static Web App per student** (`<prefix>-ui`). It calls both services from the browser over HTTPS; their base URLs are configuration (e.g. a `config.js` generated at deploy time), not hardcoded. Each service accepts cross-origin requests only from the UI's origin (CORS). Locally, compose serves the same files with a stock web-server image (e.g. `nginx:alpine` with the folder mounted, no Dockerfile); it does not count as a service container.
- **Database per service.** Each of the two services has **its own serverless SQL database** (§7) and connects with its own login, which has no access to the other

database. No service connects to another service's database; data from another service is requested over HTTP.

- **HTTP between services.** Each service locates the other through an environment variable (e.g. `CATALOG_URL=http://catalog:8000`). No hardcoded hostnames or ports.
- **Storage: SQL Server.** On Azure, **Azure SQL Database, serverless tier** (§7). Locally, one SQL Server container in compose holding one database per service (`mcr.microsoft.com/mssql/server:2022-latest`; on Apple Silicon enable Rosetta emulation in Docker Desktop). Each service reads its connection string from `DATABASE_URL`. Locally, the databases and logins are created by an idempotent script (`db/init.sql`) run by a one-off compose service; on Azure, the logins are created once during the initial setup.
- **One repository.** Each service in its own folder (`services/<name>/`) with its own dependency manifest. This replaces the single-root-manifest rule of Assignment 1.
- **System description**: an OpenAPI document per domain service and a `system.yaml` at the root (§8a).
- **Your own Dockerfiles.** The Assignment 1 template may be used as a starting point. Required: no `COPY . .`, no secrets or `.env` in the image, pinned dependencies, non-root user.
- **`docker-compose.yml`** at the repo root, bringing up the whole system with `docker compose up --build` from a clean clone.
- **CI/CD in GitHub Actions** (`.github/workflows/`), per §5.
- **Serverless deployment to Azure**, per §7: services on **Azure Container Apps (Consumption plan)**, scaling to zero when idle. Every Azure resource is billed per use; nothing runs or costs money while the app is idle. No other cloud provider.
- **No secrets in the repository**: not in source, workflows, compose file or Git history. A committed secret must be handled as an incident (§5).

1b. Free choices, justify each in `ADR.md`:

- Where the cross-seam flow lives: in a service calling the other (server side), or in the UI calling both (client side). At least one flow must cross the seam server side (§3).
- Behaviour when the other service is unavailable: fail, degrade, cache or retry (it must be handled, §3).
- Migration approach: numbered SQL files with your own runner, or a tool (Alembic, Knex, Prisma Migrate, etc.).
- Infrastructure definition: Azure CLI commands in the workflow, or IaC (Bicep or Terraform, S15). Well-understood IaC is credited under CI/CD & Deployment.
- Image registry: GitHub Container Registry (free) by default. Azure Container Registry has a fixed monthly cost and must be justified.
- Dependencies: soft cap of ~12 third-party packages per service.

1c. Out of scope: always-on compute (virtual machines, App Service plans, Container Apps dedicated workload profiles, Container Instances left running), Kubernetes/AKS, message brokers (Redis, RabbitMQ, Service Bus), background job runners, any other database (provisioned Azure SQL, Azure Database for PostgreSQL or MySQL, Cosmos DB, more than one database per service), service meshes, and a third container (frontend, gateway or other). If your design would benefit from one, document it in ADR-10 (§6).

1d. Same stack as Assignment 1. Do not rewrite services in a new language.

2. What Changes From Assignment 1

The split tests the seam described in your ADR-2.

- **Tables move.** Each table belongs to exactly one service. A foreign key across the seam becomes a plain ID on one side; referential integrity is then enforced in code, or deliberately not enforced.
- **Server-rendered pages become a static UI.** If the Assignment 1 UI was rendered by the server (templates), it becomes static pages that fetch JSON from the services. The services keep only their APIs.
- **Joins become calls.** A query that joined across domains becomes an HTTP request plus code. Many such calls suggest the seam is misplaced.
- **Seed data moves with its table.** The idempotent seeding rule (Assignment 1 §7.11) applies to every service.
- **SQLite becomes SQL Server.** Types and SQL dialect change (`INTEGER PRIMARY KEY AUTOINCREMENT` → `INT IDENTITY`, `TEXT` → `NVARCHAR`, `LIMIT` → `TOP`/`OFFSET … FETCH`). An ORM hides most of this; raw SQL must be ported. Drivers: `pymssql` or `pyodbc` (Python, `mssql+pymssql://` with SQLAlchemy), `mssql`/`tedious` (Node.js), or your ORM's SQL Server provider.

Moving the seam is allowed; ADR-6 must explain why.

3. Required Behaviour

- **Both domains work end to end** through the UI, now across two services.
- **At least one user-visible flow crosses the seam**: a request to one service that needs data from the other.
- **Partial failure is handled.** With one service stopped (`docker compose stop <service>`), the other keeps serving its own domain, and the cross-seam flow fails in a designed way (clear error, degraded response, cached data), not with a stack trace or a hung request. Every outbound call has a timeout.
- **`GET /health` on every service**, returning JSON:
  - `200`, e.g. `{"status": "ok", "db": "ok", "version": "<git sha>"}`, when the service can read and write its own database;
  - `503` otherwise.
  - Health reflects the service's own state, not the other service's.
- **Logs to stdout**: at least one line per request (method, path, status, duration).

4. Database Migrations

Assignment 1 created the schema on first boot. Once a database holds data, the schema must be changed in place, not recreated.

1. **The schema is defined by versioned migrations** committed as files (`migrations/001_init.sql`, `002_add_due_date.sql`, or your tool's equivalent), applied automatically and in order on startup.
2. **Each migration runs exactly once.** Applied migrations are recorded in the database (a `schema_migrations` table or equivalent). A second boot applies nothing.

3. **At least one migration changes a table that already holds data** (add a column with a default or backfill, rename, split a column), preserving existing rows. Dropping and recreating a table does not count.
4. **Forward-only is acceptable.** Down-migrations are not required; ADR-7 states the recovery plan for a bad migration in production.
5. **Applied migrations are never edited.** Corrections go in a new migration.
6. **Evidence on Azure.** The deployed database must contain data before the migration in (3) is deployed and retain it afterwards. The report includes the deployment run, the startup log line showing the migration, and the data before and after.

Dropping tables on boot, recreating the database on deploy, or deleting the volume to redeploy all lose data, and are checked in the code.

5. CI/CD

One or more GitHub Actions workflows that:

On every push and pull request (CI):

- Install, lint (any standard linter) and test **each** service.
- Validate `system.yaml` against `system.schema.json`, and each `openapi.yaml` against the OpenAPI specification (§8a).
- Measure coverage per service and **fail** if either service is below **70%** on its core logic.
- Build every Docker image; a failed build fails the pipeline.

On push to `main` only (CD):

- Push the images to a registry, tagged with the commit SHA (not only `latest`).
- Deploy those images to Azure, only if CI passed on the same commit.
- Build the UI and deploy it to its Static Web App (deployment token as a GitHub secret), with the services' URLs as its configuration.
- Call `/health` on the deployed services and fail the run if they do not return `200` within a set time.

CI never touches Azure SQL. Tests run against a SQL Server container (a GitHub Actions service container, or the compose stack). Every connection to the Azure database wakes it up and is billed (§7).

Secrets: the service principal credentials provided by the professor (§7) are stored as GitHub Actions secrets and read only by the workflow. The database passwords of each service user, and any other secret the system needs, are stored the same way and passed to Container Apps as secrets, never as plain environment variables.

Leaked secrets: a secret pushed to GitHub is compromised from that moment, even if a later commit deletes it, since it remains in the history. Within 24 hours of the push:

- for a credential you created (Azure keys, service principal secrets, tokens): revoke or rotate it, and update the GitHub secret that replaces it;
- for a credential provided by the professor (such as your class's service principal): notify the professor, who rotates it and sends the new one to the class;
- in both cases: remove it from the current code and record the incident in the README (what leaked, when, what was done). Rewriting Git history is optional once the credential

is revoked.

A leak handled this way is not penalised beyond the §5 grade. A final submission whose repository contains a leaked credential that was never revoked, rotated or reported is **capped at 40/100** (see Grading Criteria).

Testing, beyond Assignment 1: unit tests per service, plus **at least one integration test across the seam**, run either against the compose stack or with the other service replaced by a stub (justified in ADR-9).

6. Process Deliverables

The Assignment 1 files continue.

`ADR.md`: keep entries 1-5 unchanged; if a decision changed, set the old entry to `Status: Superseded by [N]`. Add **exactly 5 new entries**, 6-10, in the same format:

6. **The split.** Where the seam is, what crosses it, and whether ADR-2 held. If the seam moved, why.
7. **Data and migrations.** How data moved from one database to one per service, what integrity was lost at the seam and how it is compensated, and the recovery plan for a bad migration.
8. **Serverless hosting on Azure.** The Azure resources used and their cost model, and the consequences of scale-to-zero for both the containers and the database (cold starts, the database resuming from pause, connection retries, migrations at startup with a single replica). Changes needed at 100x traffic.
9. **Pipeline and testing.** What CI gates on and why, how the cross-seam test works, and what is deliberately not tested.
10. **One thing deliberately not built**, from §1c or elsewhere, and the conditions under which it would be worth building.

The new entries must span at least **3 distinct commit dates**.

`AI_USAGE.md`: same columns. Infrastructure files are included: if AI produced a workflow, Dockerfile or IaC file, log it, and explain that file in the “In my own words, how this works” column.

Commit history: at least **12 new commits** after the Assignment 1 submission, across at least **6 distinct calendar days**, with no day over 40% of them. Pushed to GitHub; push timestamps are checked.

7. Azure Deployment Contract

All students deploy to **one shared Azure subscription**.

Resource group: each class has one, already created: `rg-sdd-3a` (BCSAI 3A) or `rg-sdd-3b` (BCSAI 3B). Your IE account has Contributor access to your class’s resource group. Create all your resources there; do not create resource groups.

Credentials: the pipeline signs in to Azure with your class’s **service principal**, provided by the professor. It has Contributor on your class’s resource group only, so it can create and deploy your resources there but cannot create resource groups or assign roles. It is shared by

the whole class: store it only as a GitHub Actions secret (§5), never share it outside the class, and use it only on resources with your prefix.

Allowance per student: at most **2 Container Apps, 2 serverless SQL databases and 1 Static Web App**, exactly what §1a requires. The resource group is sized for that, so do not create more, not even temporarily.

Naming: your class's resource group is shared with your classmates, so every Azure resource you create is named with your username (the part of your IE email before `@`) as a prefix. Azure does not allow dots in most names, so replace them with hyphens: `jsmith.ieu2023@student.ie.edu` → `jsmith-ieu2023-tasks`. The SQL server and database follow the same rule (`jsmith-ieu2023-sql`, `jsmith-ieu2023-tasks-db`). Resources without your prefix may be deleted without notice. Do not modify or delete resources with another student's prefix.

The deployed system must:

1. Run both services on Azure Container Apps, Consumption plan, and the UI on one Azure Static Web App (`<prefix>-ui`), all deployed only by the pipeline. After the initial setup, nothing is created or changed manually in the portal; the initial setup is documented step by step in the README or done by IaC.
2. The public URL of the system is the Static Web App's. Both services have public HTTPS ingress, since the browser calls them, with CORS restricted to the UI's origin.
3. Store data in **one Azure SQL Database on the serverless tier per service** (§1a), all on one logical server (`<prefix>-sql`), named after the service (`<prefix>-<service>-db`): General Purpose, Gen5, minimum 0.5 vCores, maximum 1 vCore, auto-pause enabled with the shortest delay Azure allows, maximum size 2 GB, locally redundant backups. One login per service, with access to its own database only. Firewall: allow Azure services; add your own IP only while you need it.
4. **Scale to zero, and to at most one replica**: minimum replicas `0`, maximum `1` for every service, so that migrations run at startup never run twice concurrently and cost stays bounded.
5. Tolerate cold starts: the first request after idle starts the container and may find the database paused. Resuming takes up to about a minute, and connections fail meanwhile (error 40613), so connect with retries and backoff. The request is then served, after pending migrations, within the HTTP timeout. The post-deploy health check (§5) allows for this.
6. Use only pay-per-use resources: Container Apps Consumption, Azure SQL Database serverless, Static Web Apps and GHCR. Any other resource with a fixed monthly cost is justified in ADR-8.
7. Be configured entirely through environment variables and secrets set by the pipeline or IaC.
8. Remain reachable at the live URL until grades are published.

The subscription's budget is shared by the whole class. Keep a single environment and delete experimental resources as soon as they are no longer needed.

8. Documentation

8a. System description. Two kinds of machine-readable files describe the system. They are used in grading and must match the code and the deployment at the graded commit.

1. **`services/<name>/openapi.yaml`** (or `.json`) for each domain service: an OpenAPI 3 document with every endpoint, including `/health`, with methods, parameters, request and response schemas, and status codes. It may be generated by the framework (FastAPI does this natively; Flask and Express have plugins), but it is committed as a file. The endpoints one service calls on the other must exist in the other's document with the same request and response shapes.
2. **`system.yaml`** at the repository root: every service, its configuration, its calls across the seam and its Azure resources, in the format defined by the provided `system.schema.json`. Copy the schema to the repository root and start `system.yaml` with the line below; editors with YAML support (VS Code with the Red Hat YAML extension, JetBrains) then validate it and autocomplete fields.

```yaml
# yaml-language-server: $schema=./system.schema.json
student: jsmith.ieu2023
prefix: jsmith-ieu2023
public_url: https://example-name-1234.azurestaticapps.net
ui:
  path: ui
  static_web_app: jsmith-ieu2023-ui
  calls: [tasks, tags]
azure:
  resource_group: rg-sdd-3a
  container_apps_environment: jsmith-ieu2023-env
  sql_server: jsmith-ieu2023-sql
  registry: ghcr.io/jsmith
services:
  tasks:
    description: Tasks and their due dates.
    path: services/tasks
    openapi: services/tasks/openapi.yaml
    port_default: 8000
    health: /health
    migrations: services/tasks/migrations
    env:
      PORT: { default: 8000, description: Port the service listens on }
      DATABASE_URL: { secret: true, description: Connection string of the tasks
database }
      TAGS_URL: { default: "http://tags:8000", description: Base URL of the tags
service }
      TAGS_TIMEOUT_S: { default: 2, description: Timeout for calls to tags }
    calls:
      - service: tags
        url_env: TAGS_URL
        endpoints: ["GET /tags", "GET /tags/{id}"]
        timeout_s: 2
        on_failure: Tasks are listed without their tags, with a "tags unavailable"
notice.
    azure:
      container_app: jsmith-ieu2023-tasks
      sql_database: jsmith-ieu2023-tasks-db
  tags:
    # ...same fields as tasks
```

Every environment variable a service reads is listed, with its default. Secrets have no default and are marked `secret: true`. CI validates `system.yaml` against the schema and each OpenAPI document against the OpenAPI specification (§5).

8b. Report of 4-5 pages:

- **Architecture diagrams, before and after**: the Assignment 1 monolith and the split system, with the seam, the HTTP calls across it, and the database each service owns. Must match the deployment.
- **Deployment diagram**: GitHub → Actions → registry → Azure (Container Apps, SQL databases, Static Web App), naming the Azure resources created.
- **Database diagram per service**, matching the migrations.
- **Migration evidence on Azure** (§4.6).
- **Partial-failure evidence**: what a user sees when the other service is down (screenshot or response).
- **Pipeline evidence**: links to a successful CD run on `main` and to one failed run that was fixed, with the cause.
- README covering: running locally with compose; running a single service; running tests and coverage; how deployment works and how to reproduce the initial Azure setup; the live URL.
- The AI-disclosure statement required by the syllabus, as in Assignment 1.

9. Version Control

- The same individual Git repository as Assignment 1.
- Commit requirements per §6.
- Changes reach `main` through pull requests from branches, so CI runs before CD.
- **The graded version is the commit tagged `submission`**, which must be on `main`, deployed, and pushed before the deadline:

  ```
  git tag submission <commit-sha>
  git push origin submission
  ```

  The tag and the commit SHA given in the submission (Deliverables) must match. If they differ, or the tag is missing, the last commit on `main` before the deadline is graded. Commits after the tagged one are ignored.

Deliverables

- In the submission: the repository URL and the full SHA of the commit to be graded, tagged `submission` (§9)
- Git repository with the services, Dockerfiles, `docker-compose.yml`, migrations, workflows, `system.yaml`, `system.schema.json`, OpenAPI documents, and updated `ADR.md` and `AI_USAGE.md`
- The live Azure URL, in the README and the report
- Report (4-5 pages) per §8
- Attendance at the written comprehension check

Grading Criteria

Each category is scored and summed to a subtotal out of 100. As in Assignment 1, the written comprehension check is scored out of 100 and **multiplies** the subtotal (88/100 × 60% = 52.8/100).

Category	Weight	Breakdown
Split & Working System	20%	10% both domains working as separate services with their own databases (§1a, §3) · 10% cross-seam flow, timeouts and partial-failure handling (§3)
Containers & Migrations	15%	5% Dockerfiles + compose (system runs from a clean clone) · 10% versioned, data-preserving migrations, evidenced on Azure (§4)
CI/CD & Deployment	20%	10% CI gates: tests, coverage ≥70% per service, image builds (§5) · 10% CD to Azure from `main` only, SHA-tagged images, UI deployed to Static Web Apps, post-deploy health check, serverless configuration (scale to zero, pay-per-use only), no secrets in repo, live URL up (§5, §7)
Testing & Observability	10%	5% cross-seam integration test (§5) · 5% `/health` and request logs (§3)
Documentation & Report	10%	5% diagrams matching the deployment · 5% `system.yaml` and OpenAPI documents matching the code and deployment, README and evidence (§8)
Process Evidence	25%	10% `ADR.md` 6-10 · 10% `AI_USAGE.md` · 5% commit cadence & hygiene, PR flow (§6, §9)
Subtotal	**100%**	
× Written Comprehension Check	**0-100%**	Multiplies the subtotal

Leaked credentials: if the repository at the graded commit, or its history, contains a credential that was never revoked, rotated or reported to the professor (§5), the final grade is capped at **40/100**, after the comprehension check is applied.